```
$a = PHP_FLOAT_MAX;
//   1.7976931348623157E+308 = PHP_FLOAT_MAX (64bit)
$b = 1.7976931348623156E+308;

var_dump(abs(($a - $b) / $b)); // relative
var_dump(PHP_FLOAT_EPSILON);

Output:
float(1.1102230246251568E-16)
float(2.220446049250313E-16)
```

Now we're using a relative calculation, we can see that actually the two numbers are very close and within system epsilon.

There are other algorithms for float comparison, such as this one from the legendary Donald Knuth:

```
$a = PHP_FLOAT_MAX;
//   1.7976931348623157E+308 = PHP_FLOAT_MAX (64bit)
$b = 1.7976931348623156E+308;

if (
    abs($a - $b) <= PHP_FLOAT_EPSILON * $a  &&
    abs($a - $b) <= PHP_FLOAT_EPSILON * $b
) {
    print "Two largest floats are equivalent" . PHP_EOL;
}

Output:
Two largest floats are equivalent
```

There is far more to floating-point arithmetic than I have covered here, but hopefully I've given enough information to highlight the care and attention required when working with numbers like this. The next time you encounter someone tearing into PHP for its terrible bugs when working with floats, feel free to enlighten that person by laying the blame at the feet of the IEEE and hardware vendors! Every single issue with floats demonstrated here exists in C and all the languages implemented in it.